DSA Coach Agent

An AI-powered Data Structures and Algorithms (DSA) learning and assessment assistant built using Python, Streamlit, PostgreSQL, pgvector, RAG, and AI agents.

DSA Coach helps students understand DSA concepts, practice coding problems, continue conversations across sessions, search previous questions, and receive structured feedback on their solutions through an automated Rubric Generator Agent.

Project Objectives

In practical terms, DSA Coach acts as an interactive AI tutor rather than a basic chatbot: it explains concepts, retrieves relevant learning material, remembers conversation context, and evaluates submitted solutions with structured feedback.

The main objectives of DSA Coach are:

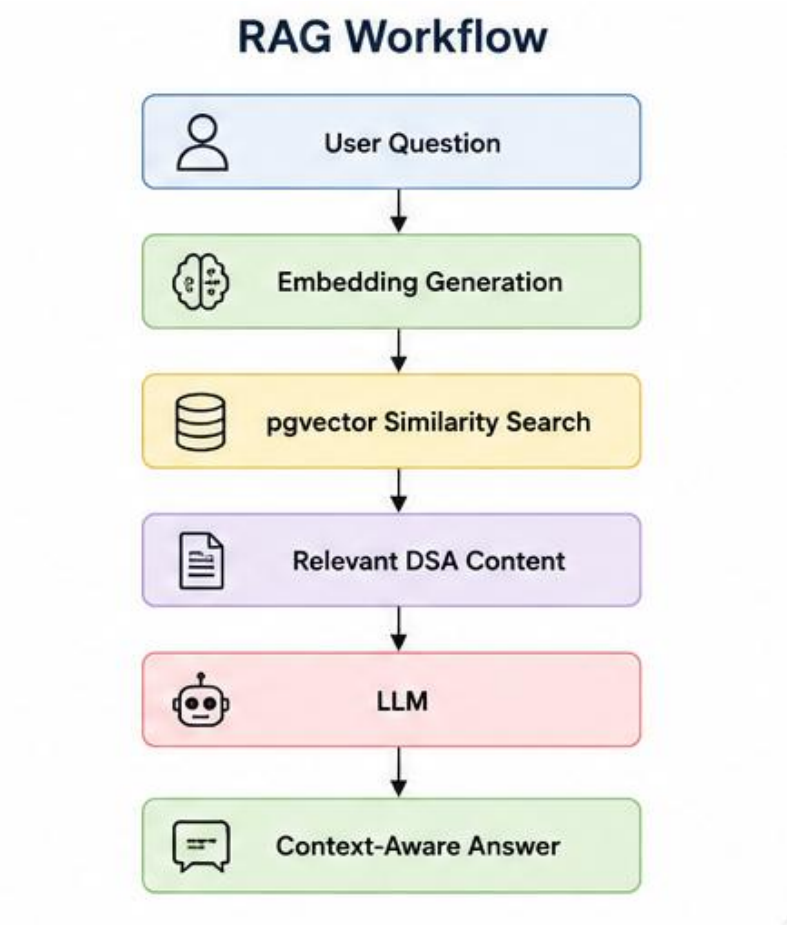
1. Help students learn DSA concepts interactively.
2. Provide context-aware explanations.
3. Use RAG to retrieve relevant learning material.
4. Maintain persistent conversation history.
5. Allow users to revisit previous questions.
6. Evaluate student solutions using generated rubrics.
7. Provide personalized feedback for improving problem-solving skills.
8. Combine conversational AI with structured DSA assessment.

Features

AI-Powered DSA Coach

- Ask questions about Data Structures and Algorithms.
- Get explanations in simple, learner-friendly language.
- Ask for Java solutions, logic, examples, dry runs, and complexity analysis.
- Continue asking follow-up questions without losing the previous context.

RAG Workflow



The system uses Retrieval-Augmented Generation (RAG) to provide relevant DSA information.

RAG helps the coach retrieve relevant information instead of relying only on the model's general knowledge.

Persistent Conversations

Users can create multiple conversations instead of keeping every question in one chat.

Example:

Recent Chats

— Two Sum Discussion

— Binary Search

— Recursion Basics

Each conversation contains its own messages and can be continued later.

Search History

The system stores previously searched questions.

Example:

Search History

— Two Sum

— 3Sum

— Binary Search

— Valid Parentheses

Selecting a previous search can take the user back to the related conversation.

Rubric Generator Agent

The Rubric Generator Agent creates an evaluation rubric based on the selected DSA problem.

Example:

Two Sum Evaluation Rubric

Correctness 40%

Time Complexity 20%

Space Complexity 15%

Code Quality 15%

Edge Case Handling 10%

The rubric can vary depending on the problem and difficulty.

Solution Evaluation

A student's solution can be evaluated against the generated rubric.

Example output:

Overall Score: 82/100

Correctness: 36/40

Time Complexity: 16/20

Space Complexity: 12/15

Code Quality: 11/15

Edge Cases: 7/10

Strengths:

- Correct approach
- Good use of HashMap
- Efficient O(n) solution

Suggestions:

- Handle the no-solution case explicitly.
- Improve variable naming.

PostgreSQL + pgvector

PostgreSQL is used for persistent application data such as:

- Conversations
- Messages
- Search history
- DSA content metadata

The pgvector extension is used for storing and searching vector

embeddings.

Technologies Used

These technologies form the core implementation stack: Python handles backend and AI logic; Streamlit provides the interface; PostgreSQL stores persistent application data; pgvector supports embedding storage and similarity search; RAG retrieves relevant DSA knowledge; the LLM generates natural-language responses and supports agent reasoning; SQL handles database operations; and Git/GitHub manages version control.

Technology Purpose

Python Backend and AI logic

Streamlit User interface

PostgreSQL Persistent database

pgvector Vector similarity search

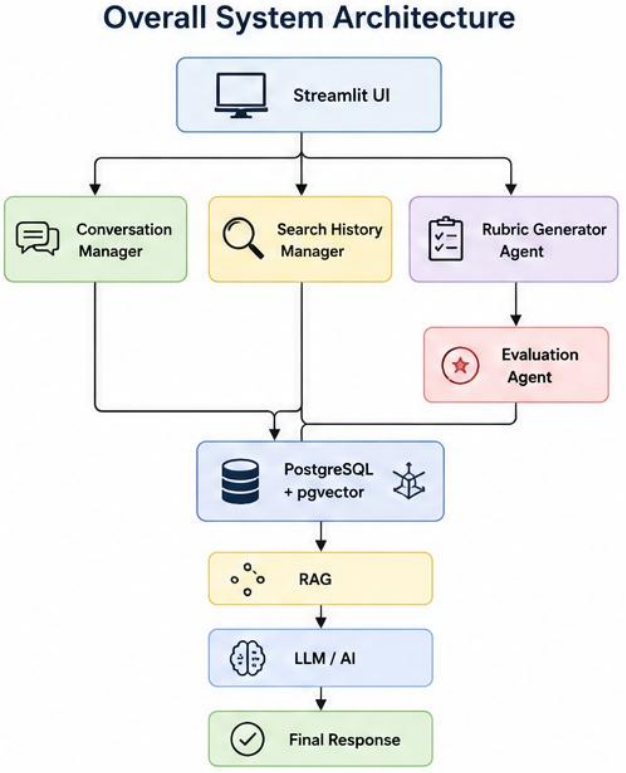
RAG Relevant knowledge retrieval

LLM Natural-language explanation and reasoning

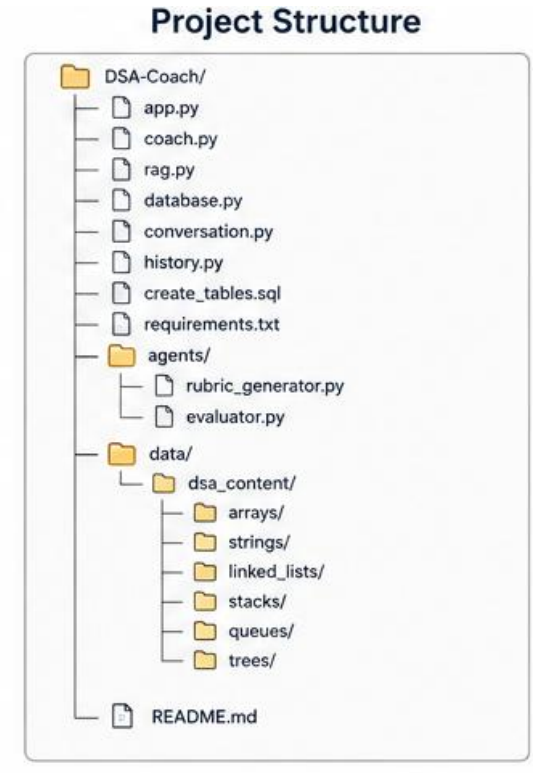
SQL Database operations

Git/GitHub Version control

System Architecture



Project Structure



DSA Coach Agent

Database Design

The project uses PostgreSQL for persistent storage.

Conversations

- Stores individual chat sessions.
- conversations
- id
- title
- created_at
- updated_at

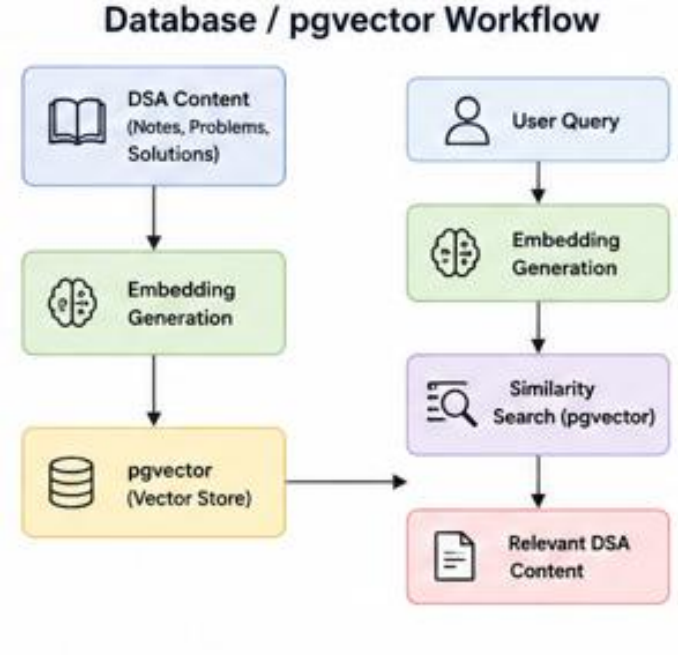
Messages

- Stores all messages belonging to a conversation.
- messages
- id
- conversation_id
- role
- content
- created_at

Search History

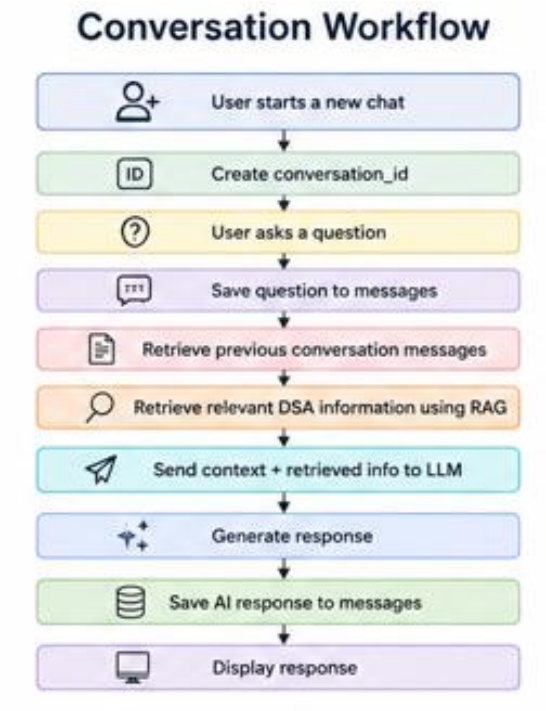
- Stores questions searched by the user.
- search_history
- id
- conversation_id
- query
- searched_at

Database / pgvector Workflow



DSA learning content is converted into embeddings and stored using **pgvector** for semantic similarity search.

Conversation Workflow



This allows users to ask follow-up questions such as:

User: Explain Two Sum.

AI: Two Sum is a problem where...

User: Give me the Java solution.

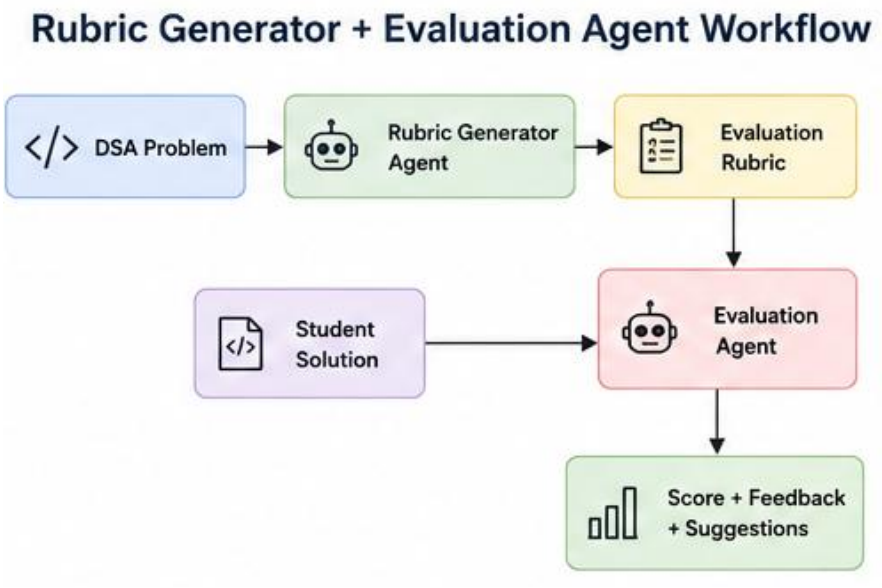
AI: Here is the Java solution...

User: Explain the second loop.

AI: The second loop...

The user does not need to repeat the original question.

Rubric Generator + Evaluation Agent Workflow



For example, a Two Sum rubric can assign weights to correctness, time complexity, space complexity, code quality, and edge-case handling. The evaluation result can then provide an overall score, strengths, and suggestions.

The evaluation can consider:

- Correctness
- Time complexity
- Space complexity
- Code quality
- Edge cases
- Problem-specific requirements

Example Use Cases

Learning
Explain Binary Search with a dry run.

Coding
Give me the Java solution for LeetCode 15.

Follow-up
Why is i < nums.length - 2?

Complexity
What is the time complexity of this solution?

Evaluation
Evaluate my solution for Two Sum.

Installation

Clone the Repository

git clone \<your-repository-url>

cd DSA-Coach

Create a Virtual Environment

python -m venv venv

Activate it on Windows:

venv\Scripts\activate

Install Dependencies

pip install -r requirements.txt

Configure PostgreSQL

Make sure PostgreSQL is installed and running.

Create a database for the project and enable pgvector.

Example:

CREATE EXTENSION IF NOT EXISTS vector;

Create Database Tables

Run the SQL commands from:

create_tables.sql

This creates the tables required for conversations, messages, search

history, and other project data.

Configure Environment Variables

Create a \.env\ file and add the required credentials.

Example:

DATABASE_URL=your_database_connection_string

API_KEY=your_api_key

Do not commit \.env\ to GitHub.

Add it to \.gitignore\:

.env

venv/

pycache/

Running the Application

Start the Streamlit application:

streamlit run app.py

The application will open in your browser.

Future Enhancements

Possible future improvements include:

- User authentication
- Personalized learning progress
- Difficulty-based problem recommendations
- Topic-wise performance tracking
- Leaderboards
- Daily DSA challenges
- Code execution and test-case validation
- Automatic detection of time and space complexity
- Learning progress dashboard
- Weak-topic identification
- Adaptive question recommendations

Team Project

The project implementation combines DSA learning, Generative AI, RAG, vector search, PostgreSQL, conversational AI, and agent-based evaluation into one application.

- Data Structures and Algorithms
- Generative AI
- Vector databases
- PostgreSQL
- Agent-based evaluation
- Conversational AI

Project Summary

DSA Coach Agent is more than a traditional chatbot. It combines RAG-based knowledge retrieval, persistent multi-turn conversations, searchable history, and an AI-powered Rubric Generator Agent to create an interactive platform for learning and evaluating DSA problem-solving skills.